

Lahore Garrison University

DHA Phase 6 Lahore



Student Name	Abdullah Khalid
Student Roll No	Fa23/BSCS/166
Section	E
Subject	Cloud Computing
Instructor Name	Sir Shahid

Cloud Computing Lab – 11

Container Orchestration using Amazon EKS

Objectives

- To deploy containerized applications
 - To manage pods and replica sets
 - To expose services externally
-

Lab Solution

A deployment was created in Amazon EKS using a YAML configuration file where the application (NGINX) was defined with **2 replicas**. When the deployment was applied, Kubernetes automatically created two pods. These pods were managed by a replica set which ensures that the required number of instances are always running.

The deployment was then verified using commands like checking running pods and deployments. At this stage, both pods were in running state which confirmed that the application was successfully deployed inside the cluster.

Scaling the Deployment

The deployment was scaled from **2 replicas to 4 replicas** using a scaling command. After executing the command, Kubernetes created two additional pods instead of restarting the existing ones.

This shows that scaling is done dynamically and represents **horizontal scaling**, where more instances are added to handle increased load.

Pod Deletion and Recovery

One of the running pods was deleted manually using a delete command. After deletion, the number of pods dropped below the desired count.

Kubernetes immediately detected this change and the replica set recreated a new pod automatically.

This behavior demonstrates the **self-healing feature** of Kubernetes, ensuring system reliability.

Exposing the Application

To make the application accessible from outside the cluster, a **LoadBalancer service** was created.

This generated an external IP address through AWS. The application could then be accessed

using a browser.

The LoadBalancer distributes traffic across all pods, improving performance and availability.

Commands Used

Shell

```
kubectl apply -f deployment.yaml
```

```
kubectl get pods
```

```
kubectl scale deployment nginx-deployment --replicas=4
```

```
kubectl delete pod <pod-name>
```

```
kubectl expose deployment nginx-deployment --type=LoadBalancer --port=80
```

Conclusion

This lab shows how Amazon EKS manages deployments, scaling, and failures automatically.

Kubernetes ensures that the required number of pods are always running and provides external access through LoadBalancer, making it efficient for real-world cloud applications.
